

PAI-RAG Agent 协议设计 Review

基于主流 Agent 协议（OpenAI Responses、Anthropic、AG-UI、A2A、MCP）的对照评审

评审日期：2026-05-14

整体评价

这份文档整体读了两遍，从协议设计的角度来看，三层接口的分层（OpenAI 兼容 / 结构化最终结果 / 完整事件流）思路是对的，和 OpenAI 自己的演进路径（Chat Completions → Responses → Assistants/Threads+Runs）几乎一致。Session / Run 的拆分、SSE 事件流、ask_user 中断、Last-Event-ID 续传这些核心机制都到位了，方向没问题。

但在事件命名、字段约定、跨请求模型选择、错误语义这些"细节"上，离一个能直接被现有生态消费的协议还差一截，下面具体说。

一、与主流事件协议对齐的问题

1.1 事件命名风格和 OpenAI Responses 不一致

这会让 SDK 适配成本变高。OpenAI Responses streaming 用的是 `response.created` / `response.output_item.added` / `response.output_text.delta` / `response.function_call_arguments.delta` / `response.completed` 这种"对象.动作"命名。你这边混用了 `reasoning.started`（动词态）、`tool.delta`（属性态）、`message.delta`、`run.completed`，每个事件的语法结构不统一。

建议统一成一种风格，并且在概念上能映射到 OpenAI 的 item / part 模型——比如 `step.created` / `step.reasoning.delta` / `step.tool_call.created` / `step.tool_call.arguments.delta` / `step.tool_call.completed` / `step.message.delta` / `run.completed`。这样写一个 adapter 转成 OpenAI 事件几乎是一对一映射。

1.2 reasoning（思维链）作为一等公民事件直接暴露需要谨慎

OpenAI Responses 把 reasoning 单独建模成 reasoning item 但默认 summary 化，原始 chain-of-thought 是不返回的；Anthropic 的 extended thinking 也只在显式 opt-in 时才以 thinking block 出现。把模型 raw thoughts 当作默认事件 push 给前端有两个风险：

- 上游模型供应商的 ToS 通常不允许原样暴露 raw CoT；
- reasoning 内容质量参差，直接渲染容易误导用户。

建议把 `reasoning.available` 改成可选 opt-in（请求里加 `include_reasoning: true` 才发），并区分 summary 和 raw 两种粒度。

1.3 tool 事件的字段命名和 OpenAI 不通用

OpenAI 用 `function_call_arguments.delta` + `function_call.completed`，参数 ID 叫 `call_id`，工具叫 `name`；你这边用 `tool_call_id` + `tool` + `arguments_delta` / `arguments_text` 并存。

`arguments_delta` 和 `arguments_text` 同时出现这点很容易让客户端混乱——一个是增量、一个是累积完整文本，**建议**只保留 `delta`，让客户端自己拼，或者改名叫 `arguments_snapshot` 让语义更清楚。

`tool_call_id` 用 `call_{turn}_{index}` 这种可解析格式也不建议——OpenAI 故意用不透明 ID（`call_ab c123`），避免客户端依赖格式做业务逻辑，将来想改成分布式 ID 时会有兼容包袱。

1.4 "幽灵字段"问题

`kind: "execute"` 这种字段在文档里只露了一次，没说还有哪些枚举值。类似的还有 `reasoning.started.hidden`、`reasoning.available.replace`，都只是表里提一笔，example 里没用到，含义没定义。这种"幽灵字段"对接入方非常不友好，要么写清要么删掉。

二、通用性问题

2.1 全局 active model 是这份设计里最值得重新考虑的点

主流 Agent 协议（OpenAI、Anthropic、Gemini、Bedrock）model 都是 per-request 的字段。你这边 `POST /v1/models/active` 改全局，写到 `memory/active_model.json`，所有新 run 都跟着变。这在生产环境里有几个硬伤：

- 多租户场景下 A 用户切模型会影响 B 用户；
- A/B 实验、灰度发布做不了；
- 客户端没法对单次请求强制指定模型；
- mtime 缓存失效在多副本部署下有竞态。

建议把 model 放回 `POST /v1/runs` 的请求体（`{"model": "qwen-max", "input": "..."}` ），`/v1/models/active` 改成"如果请求没指定 model 时的默认值"，保留全局兜底但不剥夺 per-request 覆盖能力。

2.2 Session 不能并发 run 这个约束太强

409 `session_busy` 的限制让一个 session 完全串行。OpenAI Threads 也有类似限制，但他们在 Responses API 里通过 `previous_response_id` 把会话状态拍平了，允许并发分叉。Agent 类应用经常需要"用户问 A 的时候，后台还在处理 B"，**建议**在 session 之上再抽一层 conversation，或者直接学 Responses 的 `previous_response_id`，把"哪个 run 接哪个 run 后面"做成显式 DAG，而不是 session 强串行。

2.3 没有看到 tool 定义/注册接口

Agent 用什么工具是服务端写死的，还是客户端可以传？OpenAI / Anthropic / MCP 都支持客户端在每次请求里声明 `tools: [...]`，让宿主端执行（client-side tool use）。如果你这套协议永远是 server-

managed tools，那需要在文档里写明，并解释如何扩展工具集；如果将来要支持 client-side tools，事件流里还要补 `tool.requires_action` 这类事件，现在留好扩展点比以后加破坏性变更要好。

2.4 多模态输入完全没提

`input` 字段示例都是纯字符串。OpenAI Responses 的 `input` 已经是 `string | array<content_part>`，`content_part` 支持 `text/image/file/audio`。如果你这套协议未来想接图文校验、截图分析这类场景，`input schema` 现在就该按 `array of parts` 设计，留好类型字段。

2.5 没有 MCP / A2A 的对齐讨论

MCP 是工具协议层的新标准，A2A 是 Agent 间通信的新标准。你这套是面向终端用户/UI 的协议，定位不冲突，但如果完全不考虑后端 Agent 之间用 A2A 通信、工具用 MCP server 接入，会导致整个体系是孤岛。至少应该在设计文档里说一句“工具层通过 MCP 接入”或“暂不支持，未来通过 X 扩展”。

三、兼容性问题

3.1 认证头格式不标准

示例里写 `Authorization: YmYxMDIxNDRm...`，没有 `Bearer` 前缀，也不是 `X-API-Key`。这会让任何按 OpenAI SDK 习惯写的客户端直接挂掉——OpenAI SDK 会自动加 `Bearer` 前缀。建议改成 `Authorization: Bearer <token>` 或单独的 `X-API-Key` 头，二选一。

3.2 /v1/models 返回 active_model 顶层字段是 OpenAI 不存在的扩展

OpenAI SDK 反序列化时大多会忽略多余字段所以不会崩，但有些严格的客户端（typed SDK）会报错。如果声称 OpenAI 兼容，多余字段应该放在 `metadata` 或自定义命名空间里。另外文档解释“data 里多个别名只是为了过 SDK 校验”——这等于在协议层用谎言适配 SDK，长期会有维护成本，可以考虑实际就只暴露 `active` 这一个，让 SDK 自己处理。

3.3 /v1/chat/completions 的 session 用 X-Session-Id 头传递

但 OpenAI Chat Completions 本身是无状态的，社区 SDK 默认不会保留这个头。同样，`/v1/responses` 用 `previous_response_id` / `conversation` / `session_id` 三选一更让人困惑——OpenAI 自己的 Responses 就用 `previous_response_id`，建议你这边就只支持 `previous_response_id`，把 `session_id` 作为 Responses 接口里 `metadata` 字段的一部分，而不是平级三选一。

3.4 SSE 的 [DONE] 在 ask_user 时不发送这点容易踩坑

这意味着客户端不能用“收到 `[DONE]` 即结束”这一条规则——必须在循环里同时判断 `ask_user` 事件和 `[DONE]`。建议在 `ask_user` 之后也发一个 `data: [DONE]` 或者 `data: [PAUSED]` 信号，让 SSE 连接干净关闭，客户端拿到 `ask_user` 后 POST 答案时再开新连接订阅。这样符合 SSE 长连接的常规生命周期，也省服务端连接资源。

四、易用性问题

4.1 cursor 格式前后不一致

1.2 节返回 `"cursor": "0-0"`，1.5 节断点续传里又出现 `1778640000-0`（看起来是 `timestamp_ms-seq`）。文档没说这两个是不是同一个 schema，没说 cursor 怎么从事件里取（事件 schema 里也没看到 `cursor / event_id` 字段）。**建议**每个事件都带 `id` 字段（SSE 标准就有 `id:` 行），并明确 cursor 就是最后一个事件的 id。

4.2 run 状态机有 6 个值但文档里只在 1.6 一句话带过

`started` 和 `running` 的差别是什么？`waiting_user` 期间可以 `stop` 吗？`failed` 之后能不能用同 session 再发新 run？**建议**画一张完整的状态转移图（连同 `ask_user` 那张合并），把所有合法转移和触发动作列清楚。

4.3 idempotency 完全没设计

`POST /v1/runs` 客户端超时重试就会创建重复 run。OpenAI/Stripe 都用 `Idempotency-Key` 头解决，强烈建议加上。

4.4 错误模型没有定义

`run.failed` 事件里写 `error` 字段但没 schema；4.3 校验失败时返回 `{"error": {"message", "type", "code"}}`，这个结构是好的（和 OpenAI 一致），但应该在文档开头单独建一节“错误格式”，把所有可能的 error code（`session_busy`、`invalid_request_error`、`run_not_found`、`session_not_found`、`upstream_model_error` 等）枚举出来。

4.5 usage 只在 run.completed 一次性返回

长 run 用户没法实时看到 token 消耗。OpenAI 现在支持 `stream_options: {include_usage: true}` 在每个 delta 都带累计 usage，建议跟进。

4.6 没有 webhook / 异步回调机制

SSE 长连接对手机端、Serverless 环境都不友好。生产 Agent 场景经常需要“提交后断开，跑完通过 webhook 回调”。**建议** `POST /v1/runs` 支持 `callback_url` 字段。

4.7 ask_user 缺超时和取消语义

`waiting_user` 状态下挂多久？1 小时？24 小时？永久？如果客户端崩了再也不回答，run 会一直占着 session 让后续无法继续。**建议**加 `ask_user_timeout`（默认比如 10 分钟），超时自动转 `failed` 或自动用默认 candidate 续跑。

五、其他细节

- `title: "Agent step"` 这种通用占位标题没意义，建议要么不返回，要么由模型生成实际描述。

- 文档里多次出现的 instanceId `pairec-cn-inner-khhjd7wnn1geomcirl`、Authorization token `YmYxMDIxNDRm...` 都是真实可访问的——发出去前请确认这是测试环境且 token 可以公开，不是的话需要打码。

优先级建议

如果只能改几处，按 ROI 排序，建议优先做以下几件事：

1. 把 model 改回 **per-request**
2. 统一 **cursor / event id** schema
3. 给 **ask_user** 加超时
4. Authorization 改成 **Bearer 标准**
5. 把事件命名靠拢 **OpenAI Responses 风格**

这几个改完，整套协议接入现有 OpenAI 生态 SDK 的改造量基本可以降到一个 adapter 文件，否则每个客户端都要单独适配。